

[Home](#) > [Functional Programming, SWU \(Spring 2026\)](#) > [Assignment 3](#) [Assignment 3](#) Available since 2 months · Due 2 months ago · Locked a month ago · [Grades published](#)

Assignment description

[Assignment](#)[Home](#)[Assignment description](#)[Rubric](#)[Course feedback](#)

Assignment 3

For this assignment use the [following code template](#)

Green Questions

Question 3.1

Create a function `add5 : int -> int`, that given an integer `x` adds 5 to `x`.

Create a function `mul3 : int -> int`, that given an integer `x` multiplies `x` by 3.

Create a function `add5mul3 : int -> int` that given an integer `x` first adds 5 to `x` and then multiplies the result by 3. You must use piping (`|>`).

Create a function `add5mul3_2 : int -> int` that behaves exactly like `add5mul3`. You must use function composition (`>>`).

Question 3.2

Create a function `add5_2 : ('a -> int) -> 'a -> int` that given a function `f : 'a -> int` and a value `x` of type `'a` returns `f x` plus 5.

Create a function `mul3_2 : ('a -> int) -> 'a -> int` that given a function `f : 'a -> int`

and a value x of type `'a` returns $f\ x$ multiplied by 3.

Examples:

```
> add5_2 (fun x -> x * 3) 7
val it: int = 26

> mul3_2 String.length "Hello World!!!"
val it: int = 42
```

Question 3.3

Declare a function `downto4 : (int -> 'a -> 'a) -> int -> 'a -> 'a` such that

$$\text{downto4 } f\ n\ e = \begin{cases} f\ 1\ (f\ 2\ (\dots (f\ (n-1)\ (f\ n\ e))\ \dots)) & \text{if } n > 0 \\ e & \text{if } n \leq 0 \end{cases}$$

Declare the factorial function `fac : int -> int` by use of `downto4`. The factorial function is defined as follows:

$$\begin{aligned} !0 &= 1 \\ !n &= n \cdot !(n-1) \end{aligned}$$

Use `downto4` to declare a function `range : (int -> 'a) -> int -> 'a list` that given a function `g` and an integer `n` returns the list of `[g 1, g 2, ..., g n]` if `n` is positive, and the empty list otherwise.

Challenge: An interesting solution to `downto4` makes use of partial application of the `f` function and function composition with the recursive call. Try it out if you like (it is not mandatory). The `id` function (which is just `fun x -> x`) may come in handy in that case.

Question 3.4

Create a recursive function, without using any higher-order functions, `double : int list -> int list` that given an integer list `lst` returns a list with all elements in `lst` doubled.

Create a non-recursive function, with a single use of `map`, `double_2 : int list -> int list` that behaves exactly the same as `double`.

Question 3.5

Create a recursive function, without using any higher-order functions, `stringLength : string list -> int list` that given a string list `lst` returns a list with the length of all elements in `lst`.

Create a non-recursive function, with a single use of `map`, `stringLength_2 : string list -> int list` that behaves exactly the same as `stringLength`.

Important: Take a long look at the four functions from Q3.4 and Q3.5 and see if you can spot the patterns between the recursive and the higher-order variants of the functions.

Question 3.6

Create a recursive function, without using any higher-order functions, `keepEven : int list -> int list` that given an integer list `lst` returns a list containing all elements of `lst` that are even.

Create a non-recursive function, with a single use of `filter`, `keepEven_2 : int list -> int list` that behaves exactly the same as `keepEven`.

Question 3.7

Create a recursive function, without using any higher-order functions, `keepLengthGT5 : string list -> string list` that given an string list `lst` returns a list containing all elements of `lst` that have length strictly greater than 5.

Create a non-recursive function, with a single use of `filter`, `keepLengthGT5_2 : string list -> string list` that behaves exactly the same as `keepLengthGT5`.

Important: Take a long look at the four functions from Q3.6 and Q3.7 and see if you can spot the patterns between the recursive and the higher-order variants of the functions.

Question 3.8

Create a recursive function, without using any higher-order functions, `sumPositive : int list -> int` that given an integer list `lst` sums all positive elements in `lst`.

Create a non-recursive function, with a single use of `fold`, `sumPositive_2 : int list -> int` that behaves exactly the same as `sumPositive`.

Create a non-recursive function `sumPositive_3 : int list -> int`, that behaves exactly the same as `sumPositive` and `sumPositive2`, but that removes all negative elements from `lst` using `filter` and then sums them using `fold`. Your folding function should be much simpler than the one you needed for `sumPositive2` since all odd elements are already gone.

Hint: For `sumPositive_3`, use piping (`|>`) or function composition (`>>`) to string the uses of

`filter` and `fold` together.

Yellow Questions

Question 3.9

Create a function `add5mul3_3 : ('a -> int) -> 'a -> int` that given a function `f : 'a -> int` and a value `x` of type `'a` returns `f x` plus 5 and then multiplies the result by 3. You must use the `add5_2` and `mul3_2` functions from Q3.2 and either piping or function composition.

Question 3.10

Create a non-recursive function `mergeFuns : ('a -> 'a) list -> 'a -> 'a` that given a list of functions `fs` returns a function that is the composition of all functions in `fs`.

Hint: The identity function `id` (defined as `fun x -> x`) is useful.

Examples:

```
> mergeFuns [add5; mul3] 9
val it: int = 42
```

Question 3.11

Create a non-recursive function, using a single instance of `fold`, `removeOddIdx : 'a list -> 'a list` that behaves exactly the same as `removeOddIdx` from Q2.2.

Hint: your accumulator will need to keep track if you are on an odd or an even index so some thought will have to be put into choosing an appropriate accumulator. Keep in mind that it can contain information needed for the computation

that is not part of the final result of the function.
This information needs to be initialised for the initial call to `fold` and thrown away after the `fold` completes.

Red Questions

Question 3.12

Create a function `facFuns : int -> (int -> int) list` that given an integer `x` returns a list of functions that, when combined, become the factorial function. More precisely,

```
facFuns x = [fun y -> x * y; fun y -> x - 1 * y ...; fun y -> 1 * y]
```

Hint: You can make this function recursive, but you can also use `downto4`.

Create a non-recursive function `fac_2 : int -> int`, using `mergeFuns` and `facFuns`, that given an integer `x` returns the factorial of `x`.

Question 3.13

Create a non-recursive function `weird : string list -> string` that given a list of strings `strs` removes all strings of odd length, calculates the lengths of each individual string, and appends these lengths in order into a new string.

Examples:

```
> "I feel pretty, oh so pretty, I feel pretty and witty and bright, and I pity any girl who isn't me tonight".Split(".")
|> Array.toList |> weird

val it: string = "42246442"
```

Question 3.14

Create a function `insert : 'a -> 'a list -> 'a list list` that given an element `x` and a list `xs` returns a list of lists that contains all possible ways to insert `x` into `xs`

Create a function `permutations : 'a list -> 'a list list` that given a list `lst` returns a list containing all possible permutations of `lst`.

Important: For `permutations`, the order of your resulting list is not important when compared to the examples below. As long as your list contains all possible permutations the order does not matter.

Hints:

- For `insert` the library function `List.rev` may come in handy
- for `permutations` the `insert` function and the library function `List.collect` may come in handy

Start out with writing these functions recursively, and if you want an extra challenge, move on to non-recursive higher-order variants

Examples:

```
> insert 1 [2;3;4]
val it: int list list =
  [[1; 2; 3; 4]; [2; 1; 3; 4]; [2; 3; 1; 4]; [2; 3; 4; 1]]

> permutations ["I"; "feel"; "pretty"]
val it: string list list =
  [["I"; "feel"; "pretty"]; ["feel"; "I"; "pretty"];
```

```
"pretty"]; ["feel"; "pretty"; "I"];  
... ["I"; "pretty"; "feel"]; ["pretty";  
"I"; "feel"]; ["pretty"; "feel"; "I"]]
```

© 2026 - CodeGrade ([Revanche](#)) - Made with  - [Privacy statement](#) - [Help](#)